

ObjektinisProgramavimas-2

2.0

Sugeneruota Doxygen 1.17.0

1 README	1
1.1 v1.2 pakeitimai:	1
1.1.0.1 Išvesties operatoriaus naudojimas:	2
1.1.0.2 Įvesties operatoriaus naudojimas:	2
1.1.1 Studento klasės metodų testavimo nuotraukos:	2
1.2 v1.5 pakeitimai:	2
1.2.1 Konteinerių našumo palyginimas	3
1.2.2 Pirmos strategijos tyrimo rezultatai	3
1.2.3 Antros strategijos tyrimo rezultatai	4
1.2.4 Trečios strategijos tyrimo rezultatai	4
1.2.5 Struktūrų ir objektų spartos palyginimo rezultatai	5
2 Hierarchijos Indeksas	7
2.1 Klasių hierarchija	7
3 Klasės Indeksas	9
3.1 Klasės	9
4 Failo Indeksas	11
4.1 Failai	11
5 Klasės Dokumentacija	13
5.1 Studentas Klasė	13
5.1.1 Smulkus aprašymas	15
5.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	15
5.1.2.1 Studentas() [1/6]	15
5.1.2.2 Studentas() [2/6]	15
5.1.2.3 Studentas() [3/6]	16
5.1.2.4 Studentas() [4/6]	17
5.1.2.5 ~Studentas()	17
5.1.2.6 Studentas() [5/6]	17
5.1.2.7 Studentas() [6/6]	17
5.1.3 Metodų Dokumentacija	18
5.1.3.1 addHomeworkGrade()	18
5.1.3.2 calculateFinalGrade()	18
5.1.3.3 getExamGrade()	18
5.1.3.4 getFinalGrade()	18
5.1.3.5 getHomeworkGrades()	19
5.1.3.6 getName()	19
5.1.3.7 getSurname()	19
5.1.3.8 operator=() [1/2]	19
5.1.3.9 operator=() [2/2]	20
5.1.3.10 reserveHomeworkGrades()	20
5.1.3.11 setExamGrade()	20

5.1.3.12 setFinalGrade()	20
5.1.3.13 setHomeworkGrades()	21
5.1.3.14 setName()	21
5.1.3.15 setSurname()	21
5.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	21
5.1.4.1 operator<<	21
5.1.4.2 operator>>	22
5.2 StudentasLentelei Struktūra	22
5.2.1 Smulkus aprašymas	22
5.3 Zmogus Klasė	23
5.3.1 Smulkus aprašymas	23
5.3.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	23
5.3.2.1 Zmogus() [1/2]	23
5.3.2.2 Zmogus() [2/2]	24
5.3.2.3 ~Zmogus()	25
5.3.3 Metodų Dokumentacija	25
5.3.3.1 getName()	25
5.3.3.2 getSurname()	25
6 Failo Dokumentacija	27
6.1 Studentas.h Failo Nuoroda	27
6.1.1 Smulkus aprašymas	27
6.1.2 Funkcijos Dokumentacija	27
6.1.2.1 operator<<()	27
6.2 Studentas.h	28
6.3 Zmogus.h Failo Nuoroda	29
6.3.1 Smulkus aprašymas	29
6.4 Zmogus.h	29
Rodyklė	31

skyrius 1

README

Programos aprašymas

Programa yra skirta studentų (jų akademinių pasiekimų) duomenų įvedimui, apdorojimui ir išvedimui.

Programos funkcijos:

1. Įvesti visus studentų duomenis ranka
2. Įvesti studentų vardus, pavardes, atsitiktine tvarka generuoti pažymius
3. Atsitiktinai generuoti studentų vardus, pavardes ir pažymius
4. Nuskaityti duomenis iš failo
5. Generuoti pasirinktino ilgio failą su atsitiktinai sugeneruotais studentų duomenimis

Idant būtų galima išmatuoti programos veikimo laiko priklausomybę nuo naudojamų konteinerių (**vector**, **list**, **deque**) skirstant studentus į pažangius ir nepažangius, buvo pasitelktos trys strategijos:

1. Bendro studentų konteinerio (**vector**, **list** ir **deque** tipų) išskaidymas į du naujai sukurtus to paties tipo konteinerius: pažangius ir nepažangius.
2. Bendro studentų konteinerio (**vector**, **list** ir **deque**) išskaidymas panaudojant tik nepažangių mokinių konteinerį (jei studentas nepažangus, jis perkeliamas į naują konteinerį, iš senojo ištrinamas).
3. Bendro studentų konteinerio (**vector**, **list** ir **deque**) išskaidant optimizuojant antrąją strategiją.

1.1 v1.2 pakeitimai:

Realizuotas įvesties ir išvesties operatorių perdengimas bei sukurti testai visiems studento klasės metodams bei konstruktoriams. Realizuota penkių metodų taisyklė:

1. Copy konstruktorius
2. Move konstruktorius
3. Copy assignment operatorius
4. Move assignment operatorius
5. Destruktorius

1.1.0.1 Išvesties operatoriaus naudojimas:

Naudojant operator`<<`, įmanomi du išvedimo būdai: tik su visais išvestais pažymiais bei tik su išvestu galutiniu balu. Pastarajam variantui naudojamas papildomas struct wrapper [StudentasLentelei](#):

```
Studentas studentas;
std::cout << studentas;
std::cout << StudentasLentelei{studentas};
std::ostreamstream tekstoSrautas;
tekstoSrautas << studentas;
tekstoSrautas << StudentasLentelei{studentas};
std::ofstream failoSrautas("testavimas.txt");
failoSrautas << studentas;
failoSrautas << StudentasLentelei{studentas};
```

1.1.0.2 Įvesties operatoriaus naudojimas:

Naudojant operator`>>`, pažymių kiekis nustatomas rankiniu būdu: pirmas skaičiuspo vardo ir pavardės nusako namų darbų pažymių kiekį, o pats paskutinis yra priskiriamas egzamino pažymiui.

```
Studentas studentas;
std::istreamstream tekstoSrautas("Vardenis Pavardenis 2 3 4 10");
tekstoSrautas >> studentas;
```

1.1.1 Studento klasės metodų testavimo nuotraukos:

1.2 v1.5 pakeitimai:

Pridėta abstrakti [Zmogus](#) klasė, kuri implementuojama [Studentas](#) klasėje ir pridėtas testas patikrinti abstrakčios / išvestinės klasės funkcionalumą.

Testavimo nuotrauka:

Naudojimosi instrukcija

1. Parsisiųsti ir įsirašyti [make](#) programinį paketą (tik jei naudojate Windows OS).
2. Atsidaryti savo OS terminalą
3. Klonuoti programos repozitoriją git clone <https://github.com/IvarasJoma/Objektinis-Programavimas.git>
4. Terminale rašyti žemiau esančią komandą, norint paleisti programą

```
make run
```

Testavimo apžvalga

Testavimas buvo vykdytas šioje sisteminėje aplinkoje:

Kompiuteris	Macbook M3 Pro
Procesorius	M3 Pro

Kompiuteris	Macbook M3 Pro
Branduolių skaičius	11
Operatyvios atminties talpa	18 GB LPDDR5-6400
SSD talpa ir greitis	512GB, 6200 MB/s Write, 4310 MB/s Read

1.2.1 Konteinerių našumo palyginimas

Įrašų kiekis	Konteineris	Nuskaitymas (s)	Skaičiavimas (s)	Rikiavimas (s)	Skirstymas (s)	Bendras (s)
10000000	Vector	1.46158	0.81569	0.260966	0.135772	2.67401
10000000	List	1.40333	0.842376	6.3936	1.12494	9.76424
10000000	Deque	1.24412	0.769542	0.281077	0.0778298	2.37257
1000000	Vector	0.142898	0.0756094	0.0258632	0.0129843	0.257355
1000000	List	0.134814	0.0777371	0.304793	0.0994318	0.616776
1000000	Deque	0.124166	0.0769192	0.0281017	0.00629896	0.235486
100000	Vector	0.0224329	0.00941713	0.00296424	0.00087957	0.0356938
100000	List	0.0136738	0.00784851	0.00896157	0.0016645	0.0321484
100000	Deque	0.0127949	0.0077215	0.00275661	0.000633028	0.023906
10000	Vector	0.00453481	0.0019966	0.000801319	0.000306847	0.00763957
10000	List	0.00266796	0.00149164	0.00113862	0.000238917	0.00553714
10000	Deque	0.00189549	0.00113997	0.000420417	7.90417e-05	0.00353492
1000	Vector	0.000993028	0.000160139	6.36527e-05	1.68473e-05	0.00123367
1000	List	0.000450695	0.000149583	6.34443e-05	3.52083e-05	0.000698931
1000	Deque	0.000581	0.000214389	7.65277e-05	1.73057e-05	0.000889222

1.2.2 Pirmos strategijos tyrimo rezultatai

Įrašų kiekis	Konteineris	Bendras (s)
10000000	Vector	3.14749
10000000	List	10.1786
10000000	Deque	2.94208
1000000	Vector	0.294517
1000000	List	0.640355
1000000	Deque	0.272934
100000	Vector	0.036173
100000	List	0.0341028
100000	Deque	0.0247597
10000	Vector	0.00635689
10000	List	0.00606537
10000	Deque	0.00382706

Įrašų kiekis	Konteineris	Bendras (s)
1000	Vector	0.00123278
1000	List	0.00101369
1000	Deque	0.000924639

1.2.3 Antros strategijos tyrimo rezultatai

Įrašų kiekis	Konteineris	Bendras (s)
10000000	Vector	2.82279
10000000	List	11.8649
10000000	Deque	2.35069
1000000	Vector	0.257035
1000000	List	0.81136
1000000	Deque	0.233478
100000	Vector	0.0333414
100000	List	0.0332578
100000	Deque	0.0233894
10000	Vector	0.006686
10000	List	0.00513514
10000	Deque	0.00340925
1000	Vector	0.0009305
1000	List	0.000728791
1000	Deque	0.000778236

1.2.4 Trečios strategijos tyrimo rezultatai

Įrašų kiekis	Konteineris	Bendras (s)
10000000	Vector	2.36889
10000000	List	2.55432
10000000	Deque	2.28066
1000000	Vector	0.22829
1000000	List	0.258182
1000000	Deque	0.212768
100000	Vector	0.0314771
100000	List	0.0230683
100000	Deque	0.0213107
10000	Vector	0.00756633
10000	List	0.00307204
10000	Deque	0.00224785
1000	Vector	0.0011071

Įrašų kiekis	Konteineris	Bendras (s)
1000	List	0.000699736
1000	Deque	0.000653847

1.2.5 Struktūrų ir objektų spartos palyginimo rezultatai

Versija	Optimizavimo flag'as	100k (s)	1M (s)	Failo dydis (KB)
struct	O1	0.0320068	0.234589	318
struct	O2	0.027526	0.228607	301
struct	O3	0.0255881	0.221877	317
class	O1	0.0347491	0.292031	214
class	O2	0.0330411	0.290745	213
class	O3	0.0322129	0.289138	228

Struktūrų ir objektų spartos palyginimo rezultatai

Pateikti rezultatai rodo, kad šioje programoje struct realizacija buvo spartesnė už class visais testuotais optimizavimo lygiais – tiek su 100 tūkst., tiek su 1 mln. įrašų ji demonstravo mažesnį vykdymo laiką, o didžiausias našumas pasiektas naudojant O3 optimizaciją; tuo tarpu class versija taip pat pagerėjo didinant optimizaciją, tačiau išliko lėtesnė. Tai leidžia daryti išvadą, kad struct šiuo atveju yra efektyvesnis sprendimas laiko atžvilgiu, tikėtina dėl paprastesnės struktūros ir mažesnės abstrakcijos kainos, nors skirtumai nėra kritiniai, todėl praktikoje pasirinkimas turėtų priklausyti ir nuo kodo organizavimo bei aiškumo aspektų. Kita vertus, pastebėta, kad class realizacija generuoja mažesnio dydžio vykdomuosius failus (pvz., su O2: 213 KB prieš 301 KB), todėl ji gali būti palankesnė, kai svarbus programos kompaktiškumas.

Programos testavimo nuotraukos

Išvados

1. Konteinerių tyrime std::deque ir std::vector buvo greičiausi konteineriai.
2. Studentų skaidymo į du naujus konteinerius veikė lėčiau nei perkėlimo ir ištrynimo strategija.

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

StudentasLentelei	22
Zmogus	23
Studentas	13

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	
Saugo vieno studento duomenis ir leidžia apskaičiuoti galutinį pažymį	13
StudentasLentelei	
Pagalbinė struktūra studento išvedimui lentelės formatu	22
Zmogus	
Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti	23

skyrius 4

Failo Indeksas

4.1 Failai

Visų dokumentuotų failų sąrašas su trumpais aprašymais:

Studentas.h	
Studentą aprašanti klasė, paveldinti bazinę Zmogus klasę	27
Zmogus.h	
Abstrakti bazinė žmogaus klasė	29

skyrius 5

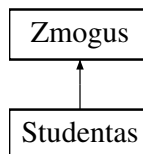
Klasės Dokumentacija

5.1 Studentas Klasė

Saugo vieno studento duomenis ir leidžia apskaičiuoti galutinį pažymį.

```
#include <Studentas.h>
```

Paveldimumo diagrama Studentas:



Vieši Metodai

- `Studentas ()`
Numatytasis konstruktorius.
- `Studentas (const std::string &name, const std::string &surname, int examGrade, const std::vector< int > &homeworkGrades, double finalGrade)`
Konstruktorius su visais pagrindiniais studento duomenimis.
- `Studentas (const char *eilute, std::size_t namuDarbuKiekis)`
Sukuria studentą iš tekstinės eilutės.
- `Studentas (bool generuotiPazymius, int ndKiekis)`
Sukuria studentą pagal generavimo nustatymus.
- `~Studentas ()`
Destruktorius.
- `Studentas (const Studentas &other)`
Kopijavimo konstruktorius.
- `Studentas & operator= (const Studentas &other)`
Kopijavimo priskyrimo operatorius.
- `Studentas (Studentas &&other) noexcept`
Perkėlimo konstruktorius.
- `Studentas & operator= (Studentas &&other) noexcept`
Perkėlimo priskyrimo operatorius.

- `std::string getName () const`
Grąžina studento vardą.
- `std::string getSurname () const`
Grąžina studento pavardę.
- `double getFinalGrade () const`
Grąžina studento galutinį pažymį.
- `int getExamGrade () const`
Grąžina studento egzamino pažymį.
- `const std::vector< int > & getHomeworkGrades () const`
Grąžina studento namų darbų pažymių sąrašą.
- `double calculateFinalGrade (char method) const`
Apskaičiuoja studento galutinį pažymį pagal pasirinktą metodą.
- `void setName (const std::string &name)`
Nustato studento vardą.
- `void setSurname (const std::string &surname)`
Nustato studento pavardę.
- `void setExamGrade (int examGrade)`
Nustato egzamino pažymį.
- `void setFinalGrade (double finalGrade)`
Nustato galutinį pažymį.
- `void setHomeworkGrades (const std::vector< int > &homeworkGrades)`
Nustato visą namų darbų pažymių sąrašą.
- `void addHomeworkGrade (int grade)`
Prideda vieną namų darbo pažymį.
- `void clearHomeworkGrades ()`
Išvalo visus namų darbų pažymius.
- `void reserveHomeworkGrades (std::size_t n)`
Rezervuoja vietą namų darbų pažymių vektoriuje.

Vieši Metodai inherited from **Zmogus**

- `Zmogus ()=default`
Numatytasis konstruktorius.
- `Zmogus (const std::string &name, const std::string &surname)`
Konstruktorius su vardu ir pavarde.
- `virtual ~Zmogus ()=0`
Grynas virtualus destruktorius.

Statiniai Vieši Atributai

- `static bool arSpausdintiDestruktoriu = false`
Nurodo, ar destruktorius turi spausdinti informacinį pranešimą.

Draugai

- `std::ostream & operator<< (std::ostream &os, const Studentas &s)`
Išveda studento duomenis į srautą.
- `std::istream & operator>> (std::istream &is, Studentas &s)`
Nuskaito studento duomenis iš įvesties srauto.

Additional Inherited Members

Apsaugoti Atributai inherited from [Zmogus](#)

- `std::string name_`
Žmogaus vardas.
- `std::string surname_`
Žmogaus pavardė.

5.1.1 Smulkus aprašymas

Saugo vieno studento duomenis ir leidžia apskaičiuoti galutinį pažymį.

Klasė paveldi abstrakčią bazinę klasę [Zmogus](#). Ji realizuoja vardo ir pavardės gavimo metodus, saugo pažymių informaciją ir pateikia metodus studento duomenims keisti bei galutiniam pažymiui apskaičiuoti.

5.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.1.2.1 Studentas() [1/6]

```
Studentas::Studentas () [inline]
```

Numatytasis konstruktorius.

Sukuria tuščią studentą su nuliniu egzamino ir galutiniu pažymiu.

5.1.2.2 Studentas() [2/6]

```
Studentas::Studentas (  
    const std::string & name,  
    const std::string & surname,  
    int examGrade,  
    const std::vector< int > & homeworkGrades,  
    double finalGrade) [inline]
```

Konstruktorius su visais pagrindiniais studento duomenimis.

Parametrai

<i>name</i>	Studento vardas.
<i>surname</i>	Studento pavardė.
<i>examGrade</i>	Egzamino pažymys.
<i>homeworkGrades</i>	Namų darbų pažymių vektorius.
<i>finalGrade</i>	Galutinis pažymys.

5.1.2.3 Studentas() [3/6]

```
Studentas::Studentas (
    const char * eilute,
    std::size_t namuDarbuKiekis)
```

Sukuria studentą iš tekstinės eilutės.

Konstruktorius naudojamas nuskaitant studentų duomenis iš failo.

Parametrai

<i>eilute</i>	Simbolių eilutė su studento duomenimis.
---------------	---

<i>namuDaruKiekis</i>	Namų darbų pažymių kiekis eilutėje.
-----------------------	-------------------------------------

5.1.2.4 Studentas() [4/6]

```
Studentas::Studentas (
    bool generuotiPazymius,
    int ndKiekis)
```

Sukuria studentą pagal generavimo nustatymus.

Parametrai

<i>generuotiPazymius</i>	Nurodo, ar pažymiai turi būti sugeneruoti automatiškai.
<i>ndKiekis</i>	Namų darbų pažymių kiekis.

5.1.2.5 ~Studentas()

```
Studentas::~~Studentas ()
```

Destruktorius.

Jei [arSpausdintiDestruktoriu](#) reikšmė yra `true`, gali būti spausdinamas informacinis pranešimas apie objekto sunaikinimą.

5.1.2.6 Studentas() [5/6]

```
Studentas::Studentas (
    const Studentas & other)
```

Kopijavimo konstruktorius.

Parametrai

<i>other</i>	Kitas studento objektas, iš kurio kopijuojami duomenys.
--------------	---

5.1.2.7 Studentas() [6/6]

```
Studentas::Studentas (
    Studentas && other) [noexcept]
```

Perkėlimo konstruktorius.

Parametrai

<i>other</i>	Kitas studento objektas, iš kurio perkeliama duomenys.
--------------	--

5.1.3 Metodų Dokumentacija

5.1.3.1 addHomeworkGrade()

```
void Studentas::addHomeworkGrade (
    int grade) [inline]
```

Prideda vieną namų darbo pažymį.

Parametrai

<i>grade</i>	Pridedamas pažymys.
--------------	---------------------

5.1.3.2 calculateFinalGrade()

```
double Studentas::calculateFinalGrade (
    char method) const
```

Apskaičiuoja studento galutinį pažymį pagal pasirinktą metodą.

Metodas paprastai nurodo, ar galutinis pažymys skaičiuojamas pagal namų darbų vidurkį, ar pagal medianą.

Parametrai

<i>method</i>	Skaiciavimo metodo simbolis.
---------------	------------------------------

Gražina

Apskaičiuotas galutinis pažymys.

5.1.3.3 getExamGrade()

```
int Studentas::getExamGrade () const [inline]
```

Gražina studento egzamino pažymį.

Gražina

Egzamino pažymys.

5.1.3.4 getFinalGrade()

```
double Studentas::getFinalGrade () const [inline]
```

Gražina studento galutinį pažymį.

Gražina

Galutinis pažymys.

5.1.3.5 getHomeworkGrades()

```
const std::vector< int > & Studentas::getHomeworkGrades () const [inline]
```

Grąžina studento namų darbų pažymių sąrašą.

Gražina

Konstantinė nuoroda į namų darbų pažymių vektorių.

5.1.3.6 getName()

```
std::string Studentas::getName () const [inline], [virtual]
```

Grąžina studento vardą.

Gražina

Studento vardas.

Realizuoja [Zmogus](#).

5.1.3.7 getSurname()

```
std::string Studentas::getSurname () const [inline], [virtual]
```

Grąžina studento pavardę.

Gražina

Studento pavardė.

Realizuoja [Zmogus](#).

5.1.3.8 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other)
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>other</i>	Kitas studento objektas, iš kurio kopijuojami duomenys.
--------------	---

Gražina

Nuoroda į esamą objektą.

5.1.3.9 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [noexcept]
```

Perkėlimo priskyrimo operatorius.

Parametrai

<i>other</i>	Kitas studento objektas, iš kurio perkeliami duomenys.
--------------	--

Gražina

Nuoroda į esamą objektą.

5.1.3.10 reserveHomeworkGrades()

```
void Studentas::reserveHomeworkGrades (  
    std::size_t n) [inline]
```

Rezervuoja vietą namų darbų pažymių vektoriuje.

Parametrai

<i>n</i>	Rezervuojamas elementų kiekis.
----------	--------------------------------

5.1.3.11 setExamGrade()

```
void Studentas::setExamGrade (  
    int examGrade) [inline]
```

Nustato egzamino pažymį.

Parametrai

<i>examGrade</i>	Naujas egzamino pažymys.
------------------	--------------------------

5.1.3.12 setFinalGrade()

```
void Studentas::setFinalGrade (  
    double finalGrade) [inline]
```

Nustato galutinį pažymį.

Parametrai

<i>finalGrade</i>	Naujas galutinis pažymys.
-------------------	---------------------------

5.1.3.13 setHomeworkGrades()

```
void Studentas::setHomeworkGrades (
    const std::vector< int > & homeworkGrades) [inline]
```

Nustato visą namų darbų pažymių sąrašą.

Parametrai

<i>homeworkGrades</i>	Naujas namų darbų pažymių vektorius.
-----------------------	--------------------------------------

5.1.3.14 setName()

```
void Studentas::setName (
    const std::string & name) [inline]
```

Nustato studento vardą.

Parametrai

<i>name</i>	Naujas vardas.
-------------	----------------

5.1.3.15 setSurname()

```
void Studentas::setSurname (
    const std::string & surname) [inline]
```

Nustato studento pavardę.

Parametrai

<i>surname</i>	Nauja pavardė.
----------------	----------------

5.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

5.1.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s) [friend]
```

Išveda studento duomenis į srautą.

Parametrai

<i>os</i>	Išvesties srautas.
-----------	--------------------

s	Studentas , kurio duomenys išvedami.
---	--

Gražina

Išvesties srautas.

5.1.4.2 operator>>

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s) [friend]
```

Nuskaityto studento duomenis iš įvesties srauto.

Parametrai

is	Įvesties srautas.
s	Studentas , į kurį įrašomi duomenys.

Gražina

Įvesties srautas.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [Studentas.h](#)
- Studentas.cpp

5.2 StudentasLentelei Struktūra

Pagalbinė struktūra studento išvedimui lentelės formatu.

```
#include <Studentas.h>
```

Vieši Atributai

- const [Studentas](#) & s
[Studentas](#), kuris bus išvedamas lentelės formatu.

5.2.1 Smulkus aprašymas

Pagalbinė struktūra studento išvedimui lentelės formatu.

Struktūra saugo konstantinę nuorodą į studentą ir naudojama specializuotam išvedimo operatoriui.

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

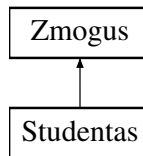
- [Studentas.h](#)

5.3 Zmogus Klasė

Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti.

```
#include <Zmogus.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- `Zmogus()`=default
Numatytasis konstruktorius.
- `Zmogus(const std::string &name, const std::string &surname)`
Konstruktorius su vardu ir pavarde.
- `virtual ~Zmogus()`=0
Grynas virtualus destruktorius.
- `virtual std::string getName()` const =0
Grąžina žmogaus vardą.
- `virtual std::string getSurname()` const =0
Grąžina žmogaus pavardę.

Apsaugoti Atributai

- `std::string name_`
Žmogaus vardas.
- `std::string surname_`
Žmogaus pavardė.

5.3.1 Smulkus aprašymas

Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti.

Klasė turi grynai virtualius metodus vardui ir pavardei gauti. Iš jos paveldi konkretesnės klasės, pavyzdžiui, `Studentas`.

5.3.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.3.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [default]
```

Numatytasis konstruktorius.

Sukuria tuščią bazinės klasės objektą.

5.3.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    const std::string & name,
    const std::string & surname) [inline]
```

Konstruktorius su vardu ir pavarde.

Parametrai

<i>name</i>	Žmogaus vardas.
-------------	-----------------

<i>surname</i>	Žmogaus pavardė.
----------------	------------------

5.3.2.3 ~Zmogus()

```
Zmogus::~~Zmogus () [inline], [pure virtual]
```

Grynas virtualus destruktorius.

Gryno virtualaus destruktoriaus apibrėžimas.

Destruktorius yra virtualus, kad išvestinių klasių objektai būtų korektiškai sunaikinami per bazinės klasės rodyklę.

Nors destruktorius yra grynai virtualus, jis vis tiek turi turėti apibrėžimą, kuris pateiktas žemiau kaip `inline`.

Grynas virtualus destruktorius privalo turėti apibrėžimą, nes jis vis tiek iškviečiamas naikinant išvestinės klasės objektą.

5.3.3 Metodų Dokumentacija

5.3.3.1 getName()

```
virtual std::string Zmogus::getName () const [pure virtual]
```

Grąžina žmogaus vardą.

Gražina

Žmogaus vardas.

Realizuota [Studentas](#).

5.3.3.2 getSurname()

```
virtual std::string Zmogus::getSurname () const [pure virtual]
```

Grąžina žmogaus pavardę.

Gražina

Žmogaus pavardė.

Realizuota [Studentas](#).

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [Zmogus.h](#)

skyrius 6

Failo Dokumentacija

6.1 Studentas.h Failo Nuoroda

Studentą aprašanti klasė, paveldinti bazinę [Zmogus](#) klasę.

```
#include <iostream>
#include <string>
#include <vector>
#include "Zmogus.h"
```

Klasės

- class [Studentas](#)
Saugo vieno studento duomenis ir leidžia apskaičiuoti galutinį pažymį.
- struct [StudentasLentelei](#)
Pagalbinė struktūra studento išvedimui lentelės formatu.

Funkcijos

- `std::ostream & operator<< (std::ostream &os, const StudentasLentelei &s)`
Išveda studentą lentelės formatu.

6.1.1 Smulkus aprašymas

Studentą aprašanti klasė, paveldinti bazinę [Zmogus](#) klasę.

Šiame faile aprašoma [Studentas](#) klasė, kuri saugo studento vardą, pavardę, egzamino pažymį, namų darbų pažymius ir galutinį pažymį. Taip pat aprašomi įvesties, išvesties, kopijavimo ir perkėlimo operatoriai.

6.1.2 Funkcijos Dokumentacija

6.1.2.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const StudentasLentelei & s)
```

Išveda studentą lentelės formatu.

Parametrai

<i>os</i>	Išvesties srautas.
-----------	--------------------

s	Pagalbinė struktūra su studento nuoroda.
---	--

Gražina

Išvesties srautas.

6.2 Studentas.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include <iostream>
00005 #include <string>
00006 #include <vector>
00007 #include "Zmogus.h"
00008
00017
00026 class Studentas : public Zmogus {
00027 private:
00029     int examGrade_;
00030
00032     std::vector<int> homeworkGrades_;
00033
00035     double finalGrade_;
00036 public:
00037     public:
00041     static bool arSpausdintiDestruktoriu;
00042
00048     Studentas() : Zmogus(), examGrade_(0), finalGrade_(0) {}
00049
00058     Studentas(const std::string& name,
00059               const std::string& surname,
00060               int examGrade,
00061               const std::vector<int>& homeworkGrades,
00062               double finalGrade)
00063         : Zmogus(name, surname),
00064           examGrade_(examGrade),
00065           homeworkGrades_(homeworkGrades),
00066           finalGrade_(finalGrade) {}
00067
00076     Studentas(const char* eilute, std::size_t namuDarbuKiekis);
00077
00083     Studentas(bool generuotiPazymius, int ndKiekis);
00090     ~Studentas();
00091
00096     Studentas(const Studentas& other);
00097
00103     Studentas& operator=(const Studentas& other);
00104
00109     Studentas(Studentas&& other) noexcept;
00110
00116     Studentas& operator=(Studentas&& other) noexcept;
00117
00122     inline std::string getName() const { return name_; }
00123
00128     inline std::string getSurname() const { return surname_; }
00133     inline double getFinalGrade() const { return finalGrade_; }
00134
00139     inline int getExamGrade() const { return examGrade_; }
00140
00145     inline const std::vector<int>& getHomeworkGrades() const { return homeworkGrades_; }
00146
00156     double calculateFinalGrade(char method) const;
00157
00162     void setName(const std::string& name) { name_ = name; }
00163
00168     void setSurname(const std::string& surname) { surname_ = surname; }
00169
00174     void setExamGrade(int examGrade) { examGrade_ = examGrade; }
00179     void setFinalGrade(double finalGrade) { finalGrade_ = finalGrade; }
00180
00185     void setHomeworkGrades(const std::vector<int>& homeworkGrades) { homeworkGrades_ = homeworkGrades; }
00186
00191     void addHomeworkGrade(int grade) { homeworkGrades_.push_back(grade); }
00192

```



```

00196     void clearHomeworkGrades() {homeworkGrades_.clear();}
00197
00202     void reserveHomeworkGrades(std::size_t n) {homeworkGrades_.reserve(n);}
00203
00210     friend std::ostream& operator<<(std::ostream& os, const Studentas& s);
00211
00218     friend std::istream& operator>>(std::istream& is, Studentas& s);
00219 };
00220
00228 struct StudentasLentelei {
00230     const Studentas& s;
00231 };
00232
00239 std::ostream& operator<<(std::ostream& os, const StudentasLentelei& s);
00240
00241 #endif

```

6.3 Zmogus.h Failo Nuoroda

Abstrakti bazinė žmogaus klasė.

```
#include <string>
```

Klasės

- class [Zmogus](#)

Abstrakti bazinė klasė žmogaus vardui ir pavardei saugoti.

6.3.1 Smulkus aprašymas

Abstrakti bazinė žmogaus klasė.

Šiame faile aprašoma abstrakti bazinė klasė [Zmogus](#). Ji saugo bendrus žmogaus duomenis: vardą ir pavardę. Klasė skirta paveldėjimui, todėl tiesiogiai [Zmogus](#) objektai nekuriami.

6.4 Zmogus.h

[Eiti į šio failo dokumentaciją.](#)

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00014
00022 class Zmogus {
00023 protected:
00027     std::string name_;
00028
00032     std::string surname_;
00033
00034 public:
00040     Zmogus() = default;
00041
00048     Zmogus(const std::string& name, const std::string& surname) : name_(name), surname_(surname) {}
00049
00059     virtual ~Zmogus() = 0;
00060
00066     virtual std::string getName() const = 0;
00067
00073     virtual std::string getSurname() const = 0;
00074 };
00075
00082 inline Zmogus::~Zmogus() {}
00083
00084 #endif

```


Rodyklė

~Studentas
 Studentas, [17](#)
~Zmogus
 Zmogus, [25](#)

addHomeworkGrade
 Studentas, [18](#)

calculateFinalGrade
 Studentas, [18](#)

getExamGrade
 Studentas, [18](#)
getFinalGrade
 Studentas, [18](#)
getHomeworkGrades
 Studentas, [18](#)
getName
 Studentas, [19](#)
 Zmogus, [25](#)
getSurname
 Studentas, [19](#)
 Zmogus, [25](#)

operator<<
 Studentas, [21](#)
 Studentas.h, [27](#)
operator>>
 Studentas, [22](#)
operator=
 Studentas, [19](#)

README, [1](#)
reserveHomeworkGrades
 Studentas, [20](#)

setExamGrade
 Studentas, [20](#)
setFinalGrade
 Studentas, [20](#)
setHomeworkGrades
 Studentas, [20](#)
setName
 Studentas, [21](#)
setSurname
 Studentas, [21](#)
Studentas, [13](#)
 ~Studentas, [17](#)
 addHomeworkGrade, [18](#)
 calculateFinalGrade, [18](#)
 getExamGrade, [18](#)
 getFinalGrade, [18](#)
 getHomeworkGrades, [18](#)
 getName, [19](#)
 getSurname, [19](#)
 operator<<, [21](#)
 operator>>, [22](#)
 operator=, [19](#)
 reserveHomeworkGrades, [20](#)
 setExamGrade, [20](#)
 setFinalGrade, [20](#)
 setHomeworkGrades, [20](#)
 setName, [21](#)
 setSurname, [21](#)
 Studentas, [15](#), [17](#)
Studentas.h, [27](#)
 operator<<, [27](#)
StudentasLentelei, [22](#)

Zmogus, [23](#)
 ~Zmogus, [25](#)
 getName, [25](#)
 getSurname, [25](#)
 Zmogus, [23](#)
Zmogus.h, [29](#)